Algorithm Interview Reference — Problems Guide

Curated Problems with Solutions and Analysis

Generated on: January 25, 2026

---

1. Union-Find (Disjoint Set Union)

Common Problem Patterns

1. Number of Islands

Problem: Given a 2D binary grid which represents a map of '1's (land) and '0's (water), return the number of islands. An island is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are surrounded by water.

Example:

```
Input: grid = [
  ["1","1","1","1","0"],
  ["1","1","0","1","0"],
  ["1","1","0","0","0"],
  ["0","0","0","0","0"]
]
Output: 1
```

```java
  public int numIslands(char[][] grid) {
      if (grid == null || grid.length == 0) return 0;

      int rows = grid.length, cols = grid[0].length;
      UnionFind uf = new UnionFind(rows * cols + 1); // +1 for water dummy node

      int[][] directions = {{0,1}, {1,0}, {0,-1}, {-1,0}};
      int waterNode = rows * cols; // Dummy node for water

      for (int i = 0; i < rows; i++) {
          for (int j = 0; j < cols; j++) {
              if (grid[i][j] == '0') {
```

```
                // Connect water cells to dummy water node
                uf.union(i * cols + j, waterNode);
            } else {
                // Connect adjacent land cells
                for (int[] dir : directions) {
                    int newRow = i + dir[0];
                    int newCol = j + dir[1];

                    if (newRow >= 0 && newRow < rows &&
                        newCol >= 0 && newCol < cols &&
                        grid[newRow][newCol] == '1') {
                        uf.union(i * cols + j, newRow * cols + newCol);
                    }
                }
            }
        }
    }

    return uf.getCount() - 1; // Subtract dummy water component
}
```

2. Friend Circles (Connected Components)

Problem: There are n people in a social network. You are given an n x n matrix isConnected where isConnected[i][j] = 1 if person i and person j are directly connected, and isConnected[i][j] = 0 otherwise. Return the total number of friend circles (connected components).

Example:

```
Input: isConnected = [[1,1,0],[1,1,0],[0,0,1]]
Output: 2
Explanation: People 0 and 1 are friends, person 2 is alone. So there are 2 friend circle
```

```
public int findCircleNum(int[][] isConnected) {
    int n = isConnected.length;
    UnionFind uf = new UnionFind(n);

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (isConnected[i][j] == 1) {
                uf.union(i, j);
            }
        }
    }

    return uf.getCount();
}
```

3. Graph Valid Tree (Cycle Detection)

Problem: You have a graph of n nodes labeled from 0 to n - 1. You are given an integer n and a list of edges where edges[i] = [ai, bi] indicates that there is an undirected edge between nodes ai and bi in the graph. Return true if the edges of the given graph make up a valid tree, and false otherwise.

Example:

```
Input: n = 5, edges = [[0,1],[0,2],[0,3],[1,4]]
Output: true
Explanation: The graph forms a valid tree.

Input: n = 5, edges = [[0,1],[1,2],[2,3],[1,3],[1,4]]
Output: false
Explanation: The graph contains a cycle.
```

```java
public boolean validTree(int n, int[][] edges) {
    // A valid tree has exactly n-1 edges and no cycles
    if (edges.length != n - 1) return false;

    UnionFind uf = new UnionFind(n);

    for (int[] edge : edges) {
        // If nodes are already connected, adding this edge creates a cycle
        if (uf.connected(edge[0], edge[1])) {
            return false;
        }
        uf.union(edge[0], edge[1]);
    }

    return uf.getCount() == 1; // Should be exactly one component
}
```

4. Accounts Merge

Problem: Given a list of accounts where each element accounts[i] is a list of strings, where the first element accounts[i][0] is a name, and the rest of the elements are emails representing emails of the account. Merge accounts that belong to the same person. Two accounts belong to the same person if there is some common email to both accounts.

Example:

```
Input: accounts = [["John","johnsmith@mail.com","john_newyork@mail.com"],
                   ["John","johnsmith@mail.com","john00@mail.com"],
                   ["Mary","mary@mail.com"],
                   ["John","johnnybravo@mail.com"]]
```

```
Output: [["John","john00@mail.com","john_newyork@mail.com","johnsmith@mail.com"],
         ["Mary","mary@mail.com"],
         ["John","johnnybravo@mail.com"]]
Explanation: The first and second John's are the same person as they have the common ema
```

```java
public List<List<String>> accountsMerge(List<List<String>> accounts) {
    Map<String, Integer> emailToIndex = new HashMap<>();
    Map<String, String> emailToName = new HashMap<>();

    int index = 0;

    // Map emails to indices and names
    for (List<String> account : accounts) {
        String name = account.get(0);
        for (int i = 1; i < account.size(); i++) {
            String email = account.get(i);
            if (!emailToIndex.containsKey(email)) {
                emailToIndex.put(email, index++);
            }
            emailToName.put(email, name);
        }
    }

    UnionFind uf = new UnionFind(index);

    // Union emails from same account
    for (List<String> account : accounts) {
        String firstEmail = account.get(1);
        for (int i = 2; i < account.size(); i++) {
            uf.union(emailToIndex.get(firstEmail), emailToIndex.get(account.get(i)))
        }
    }

    // Group emails by root
    Map<Integer, List<String>> rootToEmails = new HashMap<>();
    for (String email : emailToIndex.keySet()) {
        int root = uf.find(emailToIndex.get(email));
        rootToEmails.computeIfAbsent(root, k -> new ArrayList<>()).add(email);
    }

    // Build result
    List<List<String>> result = new ArrayList<>();
    for (List<String> emails : rootToEmails.values()) {
        Collections.sort(emails);
        List<String> account = new ArrayList<>();
        account.add(emailToName.get(emails.get(0))); // Add name
        account.addAll(emails); // Add sorted emails
        result.add(account);
    }

    return result;
}
```

Advanced Variations

Union-Find with Size Tracking

```
public class UnionFindWithSize {
    private int[] parent, size;
    private int count;

    public UnionFindWithSize(int n) {
        parent = new int[n];
        size = new int[n];
        count = n;

        for (int i = 0; i < n; i++) {
            parent[i] = i;
            size[i] = 1; // Each component initially has size 1
        }
    }

    public int find(int i) {
        if (parent[i] != i) {
            parent[i] = find(parent[i]);
        }
        return parent[i];
    }

    public void union(int i, int j) {
        int rootI = find(i);
        int rootJ = find(j);

        if (rootI != rootJ) {
            // Union by size: attach smaller component to larger
            if (size[rootI] < size[rootJ]) {
                parent[rootI] = rootJ;
                size[rootJ] += size[rootI];
            } else {
                parent[rootJ] = rootI;
                size[rootI] += size[rootJ];
            }
            count--;
        }
    }

    public int getComponentSize(int i) {
        return size[find(i)];
    }
}
```

Time & Space Complexity

Operation	Time Complexity	Space Complexity
Constructor	O(n)	O(n)
Find (with path compression)	O(α(n))*	O(1)
Union (with union by rank)	O(α(n))*	O(1)
Connected	O(α(n))*	O(1)

*α(n) is the inverse Ackermann function, which grows extremely slowly and is effectively constant for all practical purposes.

---

2. Binary Search

Common Problem Patterns

1. Range Search - First and Last Occurrence

Problem: Given an array of integers nums sorted in non-decreasing order, find the starting and ending position of a given target value. If target is not found in the array, return [-1, -1]. You must write an algorithm with O(log n) runtime complexity.

Example:

```
Input: nums = [5,7,7,8,8,10], target = 8
Output: [3,4]
Explanation: Target 8 is found at indices 3 and 4.

Input: nums = [5,7,7,8,8,10], target = 6
Output: [-1,-1]
Explanation: Target 6 is not found.
```

LeetCode 34 - Find First and Last Position of Element in Sorted Array

```
public int[] searchRange(int[] nums, int target) {
    int first = lowerBound(nums, target);

    // Check if target exists
    if (first == nums.length || nums[first] != target) {
        return new int[]{-1, -1};
    }
```

```java
        int last = upperBound(nums, target) - 1;
        return new int[]{first, last};
    }

    // Alternative approach using modified binary search
    public int[] searchRangeAlternative(int[] nums, int target) {
        int[] result = {-1, -1};

        // Find first occurrence
        result[0] = findFirst(nums, target);
        if (result[0] == -1) return result;

        // Find last occurrence
        result[1] = findLast(nums, target);

        return result;
    }

    private int findFirst(int[] nums, int target) {
        int left = 0, right = nums.length - 1;
        int result = -1;

        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] == target) {
                result = mid;
                right = mid - 1; // Continue searching left
            } else if (nums[mid] < target) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }

        return result;
    }

    private int findLast(int[] nums, int target) {
        int left = 0, right = nums.length - 1;
        int result = -1;

        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] == target) {
                result = mid;
                left = mid + 1; // Continue searching right
            } else if (nums[mid] < target) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
```

```
        return result;
    }
```

2. Search in Rotated Sorted Array

Problem: There is an integer array nums sorted in ascending order (with distinct values). Prior to being passed to your function, nums is possibly rotated at an unknown pivot index k (1 <= k < nums.length) such that the resulting array is [nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]] (0-indexed). For example, [0,1,2,4,5,6,7] might be rotated at pivot index 3 and become [4,5,6,7,0,1,2]. Given the array nums after the possible rotation and an integer target, return the index of target if it is in nums, or -1 if it is not in nums.

Example:

```
Input: nums = [4,5,6,7,0,1,2], target = 0
Output: 4

Input: nums = [4,5,6,7,0,1,2], target = 3
Output: -1
```

```
    public int search(int[] nums, int target) {
        int left = 0, right = nums.length - 1;

        while (left <= right) {
            int mid = left + (right - left) / 2;

            if (nums[mid] == target) return mid;

            // Determine which half is sorted
            if (nums[left] <= nums[mid]) {
                // Left half is sorted
                if (target >= nums[left] && target < nums[mid]) {
                    right = mid - 1; // Target in left half
                } else {
                    left = mid + 1;   // Target in right half
                }
            } else {
                // Right half is sorted
                if (target > nums[mid] && target <= nums[right]) {
                    left = mid + 1;   // Target in right half
                } else {
                    right = mid - 1;  // Target in left half
                }
            }
        }
```

```
        return -1;
    }
```

3. Find Minimum in Rotated Sorted Array

Problem: Suppose an array of length n sorted in ascending order is rotated between 1 and n times. For example, the array nums = [0,1,2,4,5,6,7] might become [4,5,6,7,0,1,2] if it was rotated 4 times. Given the sorted rotated array nums of unique elements, return the minimum element of this array.

Example:

```
Input: nums = [3,4,5,1,2]
Output: 1
Explanation: The original array was [1,2,3,4,5] rotated 3 times.

Input: nums = [4,5,6,7,0,1,2]
Output: 0
Explanation: The original array was [0,1,2,4,5,6,7] and it was rotated 4 times.
```

```java
public int findMin(int[] nums) {
    int left = 0, right = nums.length - 1;

    while (left < right) {
        int mid = left + (right - left) / 2;

        if (nums[mid] > nums[right]) {
            // Minimum is in right half
            left = mid + 1;
        } else {
            // Minimum is in left half (including mid)
            right = mid;
        }
    }

    return nums[left];
}
```

Time & Space Complexity

Operation	Time Complexity	Space Complexity
Standard Binary Search	O(log n)	O(1)
Lower/Upper Bound	O(log n)	O(1)

Search Range	O(log n)	O(1)
Binary Search on Answer	O(log(max-min) × f(n))	O(1)

Where f(n) is the complexity of the feasibility check function.

---

3. Two Pointers

Common Problem Patterns

1. Two Sum II (Sorted Array)

Problem: Given a sorted array, find two numbers that add up to a target.

```
public int[] twoSum(int[] nums, int target) {
    int left = 0, right = nums.length - 1;
    while (left < right) {
        int sum = nums[left] + nums[right];
        if (sum == target) return new int[]{left + 1, right + 1};
        else if (sum < target) left++;
        else right--;
    }
    return new int[]{-1, -1};
}
```

2. Three Sum

Problem: Find all unique triplets that sum to zero.

```
public List<List<Integer>> threeSum(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    Arrays.sort(nums);

    for (int i = 0; i < nums.length - 2; i++) {
        if (i > 0 && nums[i] == nums[i-1]) continue; // Skip duplicates

        int left = i + 1, right = nums.length - 1;
        while (left < right) {
            int sum = nums[i] + nums[left] + nums[right];
            if (sum == 0) {
                result.add(Arrays.asList(nums[i], nums[left], nums[right]));
                while (left < right && nums[left] == nums[left+1]) left++;
```

```
                while (left < right && nums[right] == nums[right-1]) right--;
                left++; right--;
            } else if (sum < 0) left++;
            else right--;
        }
    }
    return result;
}
```

3. Container With Most Water

Problem: Find two lines that together with x-axis forms a container with most water.

```
public int maxArea(int[] height) {
    int left = 0, right = height.length - 1;
    int maxWater = 0;

    while (left < right) {
        int water = Math.min(height[left], height[right]) * (right - left);
        maxWater = Math.max(maxWater, water);

        if (height[left] < height[right]) left++;
        else right--;
    }
    return maxWater;
}
```

4. Remove Duplicates from Sorted Array

Problem: Remove duplicates in-place, return new length.

```
public int removeDuplicates(int[] nums) {
    if (nums.length == 0) return 0;
    int slow = 0;
    for (int fast = 1; fast < nums.length; fast++) {
        if (nums[fast] != nums[slow]) {
            slow++;
            nums[slow] = nums[fast];
        }
    }
    return slow + 1;
}
```

5. Linked List Cycle Detection

Problem: Detect if a linked list has a cycle.

```
public boolean hasCycle(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) return true;
    }
    return false;
}
```

---

4. Sliding Window

Common Problem Patterns

1. Maximum Sum Subarray of Size K

Problem: Find maximum sum of any contiguous subarray of size k.

```
public int maxSumSubarray(int[] nums, int k) {
    int windowSum = 0, maxSum = 0;
    for (int i = 0; i < nums.length; i++) {
        windowSum += nums[i];
        if (i >= k - 1) {
            maxSum = Math.max(maxSum, windowSum);
            windowSum -= nums[i - k + 1];
        }
    }
    return maxSum;
}
```

2. Longest Substring Without Repeating Characters

Problem: Find length of longest substring without repeating characters.

```
public int lengthOfLongestSubstring(String s) {
    Map<Character, Integer> map = new HashMap<>();
    int maxLen = 0, left = 0;
```

```
        for (int right = 0; right < s.length(); right++) {
            char c = s.charAt(right);
            if (map.containsKey(c)) {
                left = Math.max(left, map.get(c) + 1);
            }
            map.put(c, right);
            maxLen = Math.max(maxLen, right - left + 1);
        }
        return maxLen;
    }
```

3. Minimum Window Substring

Problem: Find minimum window in s containing all characters of t.

```
public String minWindow(String s, String t) {
    Map<Character, Integer> need = new HashMap<>(), have = new HashMap<>();
    for (char c : t.toCharArray()) need.merge(c, 1, Integer::sum);

    int left = 0, minLen = Integer.MAX_VALUE, minStart = 0, formed = 0;

    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);
        have.merge(c, 1, Integer::sum);
        if (need.containsKey(c) && have.get(c).equals(need.get(c))) formed++;

        while (formed == need.size()) {
            if (right - left + 1 < minLen) {
                minLen = right - left + 1;
                minStart = left;
            }
            char leftChar = s.charAt(left);
            have.merge(leftChar, -1, Integer::sum);
            if (need.containsKey(leftChar) && have.get(leftChar) < need.get(leftChar
            left++;
        }
    }
    return minLen == Integer.MAX_VALUE ? "" : s.substring(minStart, minStart + minLe
}
```

4. Sliding Window Maximum

Problem: Return max sliding window of size k.

```
public int[] maxSlidingWindow(int[] nums, int k) {
    Deque<Integer> deque = new ArrayDeque<>();
    int[] result = new int[nums.length - k + 1];
```

```
        for (int i = 0; i < nums.length; i++) {
            while (!deque.isEmpty() && deque.peekFirst() < i - k + 1) deque.pollFirst();
            while (!deque.isEmpty() && nums[deque.peekLast()] < nums[i]) deque.pollLast(
            deque.offerLast(i);
            if (i >= k - 1) result[i - k + 1] = nums[deque.peekFirst()];
        }
        return result;
    }
```

---

5. Monotonic Stack

Common Problem Patterns

1. Next Greater Element

Problem: Find next greater element for each element in array.

```
public int[] nextGreaterElement(int[] nums) {
    int[] result = new int[nums.length];
    Arrays.fill(result, -1);
    Stack<Integer> stack = new Stack<>();

    for (int i = 0; i < nums.length; i++) {
        while (!stack.isEmpty() && nums[stack.peek()] < nums[i]) {
            result[stack.pop()] = nums[i];
        }
        stack.push(i);
    }
    return result;
}
```

2. Daily Temperatures

Problem: Find days until warmer temperature for each day.

```
public int[] dailyTemperatures(int[] temperatures) {
    int[] result = new int[temperatures.length];
    Stack<Integer> stack = new Stack<>();

    for (int i = 0; i < temperatures.length; i++) {
        while (!stack.isEmpty() && temperatures[stack.peek()] < temperatures[i]) {
```

```
            int prevDay = stack.pop();
            result[prevDay] = i - prevDay;
        }
        stack.push(i);
    }
    return result;
}
```

3. Largest Rectangle in Histogram

Problem: Find largest rectangle area in histogram.

```
public int largestRectangleArea(int[] heights) {
    Stack<Integer> stack = new Stack<>();
    int maxArea = 0;

    for (int i = 0; i <= heights.length; i++) {
        int h = (i == heights.length) ? 0 : heights[i];
        while (!stack.isEmpty() && heights[stack.peek()] > h) {
            int height = heights[stack.pop()];
            int width = stack.isEmpty() ? i : i - stack.peek() - 1;
            maxArea = Math.max(maxArea, height * width);
        }
        stack.push(i);
    }
    return maxArea;
}
```

4. Trapping Rain Water

Problem: Calculate trapped rain water between bars.

```
public int trap(int[] height) {
    Stack<Integer> stack = new Stack<>();
    int water = 0;

    for (int i = 0; i < height.length; i++) {
        while (!stack.isEmpty() && height[stack.peek()] < height[i]) {
            int bottom = stack.pop();
            if (stack.isEmpty()) break;
            int width = i - stack.peek() - 1;
            int h = Math.min(height[i], height[stack.peek()]) - height[bottom];
            water += width * h;
        }
        stack.push(i);
    }
```

```
        return water;
    }
```

---

6. Intervals

Common Problem Patterns

1. Merge Intervals

Problem: Merge all overlapping intervals.

```
public int[][] merge(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
    List<int[]> result = new ArrayList<>();

    for (int[] interval : intervals) {
        if (result.isEmpty() || result.get(result.size() - 1)[1] < interval[0]) {
            result.add(interval);
        } else {
            result.get(result.size() - 1)[1] =
                Math.max(result.get(result.size() - 1)[1], interval[1]);
        }
    }
    return result.toArray(new int[result.size()][]);
}
```

2. Insert Interval

Problem: Insert new interval into sorted non-overlapping intervals.

```
public int[][] insert(int[][] intervals, int[] newInterval) {
    List<int[]> result = new ArrayList<>();
    int i = 0;

    // Add all intervals before newInterval
    while (i < intervals.length && intervals[i][1] < newInterval[0]) {
        result.add(intervals[i++]);
    }
    // Merge overlapping intervals
    while (i < intervals.length && intervals[i][0] <= newInterval[1]) {
        newInterval[0] = Math.min(newInterval[0], intervals[i][0]);
```

```
            newInterval[1] = Math.max(newInterval[1], intervals[i][1]);
            i++;
        }
        result.add(newInterval);
        // Add remaining intervals
        while (i < intervals.length) result.add(intervals[i++]);

        return result.toArray(new int[result.size()][]);
    }
```

3. Meeting Rooms II (Min Rooms Required)

Problem: Find minimum meeting rooms required.

```
public int minMeetingRooms(int[][] intervals) {
    int[] starts = new int[intervals.length];
    int[] ends = new int[intervals.length];

    for (int i = 0; i < intervals.length; i++) {
        starts[i] = intervals[i][0];
        ends[i] = intervals[i][1];
    }
    Arrays.sort(starts);
    Arrays.sort(ends);

    int rooms = 0, endPtr = 0;
    for (int start : starts) {
        if (start < ends[endPtr]) rooms++;
        else endPtr++;
    }
    return rooms;
}
```

4. Non-Overlapping Intervals

Problem: Remove minimum intervals to make rest non-overlapping.

```
public int eraseOverlapIntervals(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> a[1] - b[1]); // Sort by end time
    int count = 0, prevEnd = Integer.MIN_VALUE;

    for (int[] interval : intervals) {
        if (interval[0] >= prevEnd) {
            prevEnd = interval[1];
        } else {
            count++; // Remove this interval
        }
    }
```

```
        return count;
    }
```

---

7. Top-K & Heaps

Common Problem Patterns

1. Top-K Frequent Elements

Problem: Given an integer array nums and an integer k, return the k most frequent elements.

Example:

```
Input: nums = [1,1,1,2,2,3], k = 2
Output: [1,2]
```

```
public int[] topKFrequent(int[] nums, int k) {
    // Count frequency
    Map<Integer, Integer> frequencyMap = new HashMap<>();
    for (int num : nums) {
        frequencyMap.put(num, frequencyMap.getOrDefault(num, 0) + 1);
    }

    // Use min-heap to track top k frequent elements
    PriorityQueue<int[]> minHeap = new PriorityQueue<>((a, b) -> a[1] - b[1]);

    for (Map.Entry<Integer, Integer> entry : frequencyMap.entrySet()) {
        minHeap.offer(new int[]{entry.getKey(), entry.getValue()});

        if (minHeap.size() > k) {
            minHeap.poll();
        }
    }

    // Extract results
    int[] result = new int[k];
    for (int i = k - 1; i >= 0; i--) {
        result[i] = minHeap.poll()[0];
    }

    return result;
}
```

2. Kth Largest Element in Array

Problem: Find the kth largest element in an unsorted array.

Example:

```
Input: nums = [3,2,1,5,6,4], k = 2
Output: 5
```

```java
public int findKthLargest(int[] nums, int k) {
    PriorityQueue<Integer> minHeap = new PriorityQueue<>();

    for (int num : nums) {
        minHeap.offer(num);

        if (minHeap.size() > k) {
            minHeap.poll();
        }
    }

    return minHeap.peek();
}

// Alternative: Quick Select approach
public int findKthLargestQuickSelect(int[] nums, int k) {
    return quickSelect(nums, 0, nums.length - 1, nums.length - k);
}

private int quickSelect(int[] nums, int left, int right, int targetIndex) {
    int pivotIndex = partition(nums, left, right);

    if (pivotIndex == targetIndex) {
        return nums[pivotIndex];
    } else if (pivotIndex < targetIndex) {
        return quickSelect(nums, pivotIndex + 1, right, targetIndex);
    } else {
        return quickSelect(nums, left, pivotIndex - 1, targetIndex);
    }
}

private int partition(int[] nums, int left, int right) {
    int pivot = nums[right];
    int i = left;

    for (int j = left; j < right; j++) {
        if (nums[j] <= pivot) {
            swap(nums, i, j);
            i++;
        }
    }

    swap(nums, i, right);
```

```
        return i;
    }
```

3. Find Median from Data Stream

Problem: Design a data structure that supports adding numbers and finding median efficiently.

```
class MedianFinder {
    private PriorityQueue<Integer> maxHeap; // smaller half
    private PriorityQueue<Integer> minHeap; // larger half

    public MedianFinder() {
        maxHeap = new PriorityQueue<>(Collections.reverseOrder());
        minHeap = new PriorityQueue<>();
    }

    public void addNum(int num) {
        // Add to maxHeap first
        maxHeap.offer(num);

        // Ensure maxHeap.top <= minHeap.top
        minHeap.offer(maxHeap.poll());

        // Balance heaps (maxHeap can have at most 1 extra element)
        if (minHeap.size() > maxHeap.size()) {
            maxHeap.offer(minHeap.poll());
        }
    }

    public double findMedian() {
        if (maxHeap.size() > minHeap.size()) {
            return maxHeap.peek();
        } else {
            return (maxHeap.peek() + minHeap.peek()) / 2.0;
        }
    }
}
```

4. Merge K Sorted Lists

Problem: Merge k sorted linked lists and return it as one sorted list.

Example:

```
Input: lists = [[1,4,5],[1,3,4],[2,6]]
Output: [1,1,2,3,4,4,5,6]
```

```
public ListNode mergeKLists(ListNode[] lists) {
    if (lists == null || lists.length == 0) return null;

    PriorityQueue<ListNode> minHeap = new PriorityQueue<>((a, b) -> a.val - b.val);

    // Add first node of each list to heap
    for (ListNode head : lists) {
        if (head != null) {
            minHeap.offer(head);
        }
    }

    ListNode dummy = new ListNode(0);
    ListNode current = dummy;

    while (!minHeap.isEmpty()) {
        ListNode smallest = minHeap.poll();
        current.next = smallest;
        current = current.next;

        if (smallest.next != null) {
            minHeap.offer(smallest.next);
        }
    }

    return dummy.next;
}
```

5. Task Scheduler

Problem: Given tasks and cooldown time, find minimum intervals needed.

Example:

```
Input: tasks = ["A","A","A","B","B","B"], n = 2
Output: 8
Explanation: A -> B -> idle -> A -> B -> idle -> A -> B
```

```
public int leastInterval(char[] tasks, int n) {
    // Count task frequencies
    int[] frequencies = new int[26];
    for (char task : tasks) {
        frequencies[task - 'A']++;
    }

    // Use max-heap to track most frequent tasks
    PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder())
    for (int freq : frequencies) {
```

```
            if (freq > 0) {
                maxHeap.offer(freq);
            }
        }

        int time = 0;
        Queue<int[]> cooldown = new LinkedList<>(); // [frequency, available_time]

        while (!maxHeap.isEmpty() || !cooldown.isEmpty()) {
            time++;

            // Add tasks back from cooldown
            if (!cooldown.isEmpty() && cooldown.peek()[1] == time) {
                maxHeap.offer(cooldown.poll()[0]);
            }

            // Execute most frequent task
            if (!maxHeap.isEmpty()) {
                int freq = maxHeap.poll() - 1;
                if (freq > 0) {
                    cooldown.offer(new int[]{freq, time + n + 1});
                }
            }
        }

        return time;
    }
```

Advanced Variations

Sliding Window Maximum

```
    public int[] maxSlidingWindow(int[] nums, int k) {
        if (nums == null || nums.length == 0) return new int[0];

        PriorityQueue<int[]> maxHeap = new PriorityQueue<>((a, b) -> b[0] - a[0]);
        int[] result = new int[nums.length - k + 1];

        for (int i = 0; i < nums.length; i++) {
            maxHeap.offer(new int[]{nums[i], i});

            // Remove elements outside window
            while (!maxHeap.isEmpty() && maxHeap.peek()[1] <= i - k) {
                maxHeap.poll();
            }

            // Record maximum for current window
            if (i >= k - 1) {
                result[i - k + 1] = maxHeap.peek()[0];
            }
        }
```

```
        return result;
    }
```

K Closest Points to Origin

```
public int[][] kClosest(int[][] points, int k) {
    PriorityQueue<int[]> maxHeap = new PriorityQueue<>((a, b) ->
        (b[0] * b[0] + b[1] * b[1]) - (a[0] * a[0] + a[1] * a[1]));

    for (int[] point : points) {
        maxHeap.offer(point);

        if (maxHeap.size() > k) {
            maxHeap.poll();
        }
    }

    int[][] result = new int[k][2];
    for (int i = k - 1; i >= 0; i--) {
        result[i] = maxHeap.poll();
    }

    return result;
}
```

Meeting Rooms II

```
public int minMeetingRooms(int[][] intervals) {
    if (intervals == null || intervals.length == 0) return 0;

    Arrays.sort(intervals, (a, b) -> a[0] - b[0]);

    PriorityQueue<Integer> minHeap = new PriorityQueue<>();

    for (int[] interval : intervals) {
        // Remove meetings that have ended
        if (!minHeap.isEmpty() && minHeap.peek() <= interval[0]) {
            minHeap.poll();
        }

        minHeap.offer(interval[1]);
    }

    return minHeap.size();
}
```

Interview Tips

Pattern Recognition

- **Top-K problems**: Use heap of size K to avoid sorting entire array
- **Streaming data**: Heaps maintain running maximum/minimum efficiently
- **Merge operations**: Priority queue orders elements from multiple sources
- **Scheduling problems**: Heaps track earliest available times/deadlines

Common Mistakes

- **Wrong heap type**: Using max-heap when min-heap is needed (or vice versa)
- **Heap size management**: Not maintaining fixed size for top-K problems
- **Comparator errors**: Incorrect custom comparison functions
- **Empty heap access**: Not checking if heap is empty before peek/poll

Implementation Variations

```
// Custom object in heap
class Task {
    String name;
    int priority;

    Task(String name, int priority) {
        this.name = name;
        this.priority = priority;
    }
}

PriorityQueue<Task> taskQueue = new PriorityQueue<>((a, b) -> b.priority - a.priorit

// Multiple criteria comparison
PriorityQueue<int[]> complexHeap = new PriorityQueue<>((a, b) -> {
    if (a[0] != b[0]) return a[0] - b[0]; // First by first element
    return a[1] - b[1]; // Then by second element
});

// Lazy deletion for sliding window
Map<Integer, Integer> toRemove = new HashMap<>();
while (!maxHeap.isEmpty() && toRemove.containsKey(maxHeap.peek())) {
    int val = maxHeap.poll();
    toRemove.put(val, toRemove.get(val) - 1);
    if (toRemove.get(val) == 0) toRemove.remove(val);
}
```

Time & Space Complexity

Operation	Time Complexity	Space Complexity	Notes
Insert	O(log n)	O(1)	Single heap operation
Extract Min/Max	O(log n)	O(1)	Single heap operation
Peek	O(1)	O(1)	No modification
Top-K Elements	O(n log k)	O(k)	k-sized heap
Merge K Lists	O(n log k)	O(k)	k nodes in heap
Find Median (stream)	O(log n)	O(n)	Two heaps
Quick Select	O(n) average, $O(n^2)$ worst	O(1)	In-place partition

---

8. Greedy Algorithms

Common Problem Patterns

1. Gas Station Problem

Problem: There are n gas stations along a circular route. Determine if you can complete the circuit.

Example:

```
Input: gas = [1,2,3,4,5], cost = [3,4,5,1,2]
Output: 3 (start at station 3)
```

```
public int canCompleteCircuit(int[] gas, int[] cost) {
    int totalGas = 0, totalCost = 0;
    int currentGas = 0, startStation = 0;
```

```
        for (int i = 0; i < gas.length; i++) {
            totalGas += gas[i];
            totalCost += cost[i];
            currentGas += gas[i] - cost[i];

            // If we can't reach next station, try starting from next station
            if (currentGas < 0) {
                startStation = i + 1;
                currentGas = 0;
            }
        }

        // Check if total gas is enough
        return totalGas >= totalCost ? startStation : -1;
    }
```

2. Minimum Number of Arrows

Problem: Burst balloons with minimum arrows. Each balloon has start and end coordinates.

Example:

```
Input: points = [[10,16],[2,8],[1,6],[7,12]]
Output: 2 (one arrow at [6,7] area, another at [12,16])
```

```
    public int findMinArrowShots(int[][] points) {
        if (points.length == 0) return 0;

        // Sort by end coordinate
        Arrays.sort(points, (a, b) -> Integer.compare(a[1], b[1]));

        int arrows = 1;
        int arrowPosition = points[0][1];

        for (int i = 1; i < points.length; i++) {
            // If balloon starts after arrow position, need new arrow
            if (points[i][0] > arrowPosition) {
                arrows++;
                arrowPosition = points[i][1];
            }
        }

        return arrows;
    }
```

3. Task Scheduler

Problem: Given tasks and cooldown time n, find minimum time to finish all tasks.

Example:

```
Input: tasks = ["A","A","A","B","B","B"], n = 2
Output: 8
Explanation: A -> B -> idle -> A -> B -> idle -> A -> B
```

```java
public int leastInterval(char[] tasks, int n) {
    // Count task frequencies
    int[] frequencies = new int[26];
    for (char task : tasks) {
        frequencies[task - 'A']++;
    }

    Arrays.sort(frequencies);

    // Most frequent task
    int maxFreq = frequencies[25];

    // Number of idle slots needed for most frequent task
    int idleSlots = (maxFreq - 1) * n;

    // Fill idle slots with other tasks
    for (int i = 24; i >= 0 && idleSlots > 0; i--) {
        idleSlots -= Math.min(frequencies[i], maxFreq - 1);
    }

    idleSlots = Math.max(0, idleSlots);

    return tasks.length + idleSlots;
}
```

4. Candy Distribution

Problem: Children sit in a line, each has a rating. Give candies such that higher-rated children get more than neighbors.

Example:

```
Input: ratings = [1,0,2]
Output: 5
Explanation: Give [2,1,2] candies respectively.
```

```java
public int candy(int[] ratings) {
    int n = ratings.length;
```

```
        int[] candies = new int[n];
        Arrays.fill(candies, 1); // Everyone gets at least 1 candy

        // Left to right: ensure right neighbor with higher rating gets more
        for (int i = 1; i < n; i++) {
            if (ratings[i] > ratings[i - 1]) {
                candies[i] = candies[i - 1] + 1;
            }
        }

        // Right to left: ensure left neighbor with higher rating gets more
        for (int i = n - 2; i >= 0; i--) {
            if (ratings[i] > ratings[i + 1]) {
                candies[i] = Math.max(candies[i], candies[i + 1] + 1);
            }
        }

        int totalCandies = 0;
        for (int candy : candies) {
            totalCandies += candy;
        }

        return totalCandies;
    }
```

5. Remove K Digits

Problem: Remove k digits from number to make it smallest possible.

Example:

```
Input: num = "1432219", k = 3
Output: "1219"
```

```
    public String removeKdigits(String num, int k) {
        Stack<Character> stack = new Stack<>();

        for (char digit : num.toCharArray()) {
            // Remove larger digits from stack
            while (!stack.isEmpty() && k > 0 && stack.peek() > digit) {
                stack.pop();
                k--;
            }
            stack.push(digit);
        }

        // Remove remaining digits from end
        while (k > 0) {
            stack.pop();
            k--;
```

```
        }

        // Build result
        StringBuilder result = new StringBuilder();
        while (!stack.isEmpty()) {
            result.append(stack.pop());
        }
        result.reverse();

        // Remove leading zeros
        while (result.length() > 1 && result.charAt(0) == '0') {
            result.deleteCharAt(0);
        }

        return result.length() == 0 ? "0" : result.toString();
    }
```

Advanced Variations

Minimum Spanning Tree (Kruskal's Algorithm)

```
import java.util.*;

public class KruskalMST {
    static class Edge implements Comparable<Edge> {
        int src, dest, weight;

        Edge(int src, int dest, int weight) {
            this.src = src;
            this.dest = dest;
            this.weight = weight;
        }

        @Override
        public int compareTo(Edge other) {
            return this.weight - other.weight;
        }
    }

    static class UnionFind {
        int[] parent, rank;

        UnionFind(int n) {
            parent = new int[n];
            rank = new int[n];
            for (int i = 0; i < n; i++) {
                parent[i] = i;
            }
        }

        int find(int x) {
```

```
            if (parent[x] != x) {
                parent[x] = find(parent[x]);
            }
            return parent[x];
        }

        boolean union(int x, int y) {
            int rootX = find(x);
            int rootY = find(y);

            if (rootX == rootY) return false;

            if (rank[rootX] < rank[rootY]) {
                parent[rootX] = rootY;
            } else if (rank[rootX] > rank[rootY]) {
                parent[rootY] = rootX;
            } else {
                parent[rootY] = rootX;
                rank[rootX]++;
            }

            return true;
        }
    }

    public static List<Edge> kruskalMST(List<Edge> edges, int vertices) {
        Collections.sort(edges); // Greedy: sort by weight

        UnionFind uf = new UnionFind(vertices);
        List<Edge> mst = new ArrayList<>();

        for (Edge edge : edges) {
            if (uf.union(edge.src, edge.dest)) {
                mst.add(edge);

                if (mst.size() == vertices - 1) break;
            }
        }

        return mst;
    }
}
```

Stock Buy Sell with Transaction Fee

```
public class StockWithFee {

    /**
     * Best Time to Buy and Sell Stock with Transaction Fee
     */
    public int maxProfit(int[] prices, int fee) {
        int buy = -prices[0]; // Max profit when holding stock
```

```
        int sell = 0;         // Max profit when not holding stock

        for (int i = 1; i < prices.length; i++) {
            int prevBuy = buy;
            int prevSell = sell;

            // Either keep holding or buy today
            buy = Math.max(prevBuy, prevSell - prices[i]);

            // Either keep not holding or sell today (pay fee)
            sell = Math.max(prevSell, prevBuy + prices[i] - fee);
        }

        return sell;
    }
}
```

Interview Tips

Pattern Recognition

- **Optimization problems**: Look for greedy choice property
- **Sorting first**: Many greedy algorithms require sorting input
- **Local optimum**: Each step makes locally best choice
- **No backtracking**: Once choice is made, never reconsider

Common Mistakes

- **Assuming greedy works**: Always verify greedy choice property
- **Wrong sorting criteria**: Choose correct parameter to sort by
- **Missing edge cases**: Empty input, single element, all same values
- **Not proving optimality**: Greedy might give suboptimal results

Implementation Variations

```
// Template for interval problems
public static int greedyIntervals(int[][] intervals) {
    if (intervals.length == 0) return 0;

    // Sort by appropriate criteria (end time, start time, etc.)
    Arrays.sort(intervals, (a, b) -> a[1] - b[1]);

    int count = 1;
    int lastEnd = intervals[0][1];

    for (int i = 1; i < intervals.length; i++) {
```

```
            if (intervals[i][0] >= lastEnd) {
                count++;
                lastEnd = intervals[i][1];
            }
        }

        return count;
    }

    // Template for choosing maximum/minimum
    public static int greedyChoice(int[] nums) {
        Arrays.sort(nums); // Or custom sorting

        int result = 0;
        for (int i = 0; i < nums.length; i++) {
            // Make greedy choice based on problem
            if (/* condition */) {
                result += nums[i];
            }
        }

        return result;
    }
```

Time & Space Complexity

Algorithm	Time Complexity	Space Complexity	Notes
Activity Selection	O(n log n)	O(1)	Sorting dominates
Fractional Knapsack	O(n log n)	O(1)	Sort by ratio
Huffman Coding	O(n log n)	O(n)	Priority queue operations
Jump Game	O(n)	O(1)	Single pass
Gas Station	O(n)	O(1)	Single pass
MST (Kruskal)	O(E log E)	O(V)	Sort edges, Union-Find
Task Scheduler	O(n)	O(1)	Counting sort

---

9. Dynamic Programming

Common Problem Patterns

1. 0/1 Knapsack

Problem: Given weights and values of n items, put items in a knapsack of capacity W to maximize total value.

```
public int knapsack(int[] weights, int[] values, int W) {
    int n = weights.length;
    int[] dp = new int[W + 1]; // Space optimized

    for (int i = 0; i < n; i++) {
        // Traverse backwards to avoid using same item twice
        for (int w = W; w >= weights[i]; w--) {
            dp[w] = Math.max(dp[w], dp[w - weights[i]] + values[i]);
        }
    }
    return dp[W];
}
```

2. Longest Increasing Subsequence (LIS)

Problem: Find the length of the longest strictly increasing subsequence.

```
// O(n log n) solution using binary search
public int lengthOfLIS(int[] nums) {
    List<Integer> tails = new ArrayList<>();

    for (int num : nums) {
        int pos = Collections.binarySearch(tails, num);
        if (pos < 0) pos = -(pos + 1);

        if (pos == tails.size()) {
            tails.add(num);
        } else {
            tails.set(pos, num);
        }
    }
    return tails.size();
}

// O(n²) classic DP solution
public int lengthOfLIS_DP(int[] nums) {
    int n = nums.length;
    int[] dp = new int[n];
    Arrays.fill(dp, 1);
    int maxLen = 1;
```

```
        for (int i = 1; i < n; i++) {
            for (int j = 0; j < i; j++) {
                if (nums[j] < nums[i]) {
                    dp[i] = Math.max(dp[i], dp[j] + 1);
                }
            }
            maxLen = Math.max(maxLen, dp[i]);
        }
        return maxLen;
    }
```

3. Edit Distance (Levenshtein)

Problem: Find minimum operations (insert, delete, replace) to convert word1 to word2.

```
public int minDistance(String word1, String word2) {
    int m = word1.length(), n = word2.length();
    int[][] dp = new int[m + 1][n + 1];

    // Base cases
    for (int i = 0; i <= m; i++) dp[i][0] = i;
    for (int j = 0; j <= n; j++) dp[0][j] = j;

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (word1.charAt(i - 1) == word2.charAt(j - 1)) {
                dp[i][j] = dp[i - 1][j - 1];
            } else {
                dp[i][j] = 1 + Math.min(dp[i - 1][j - 1],  // Replace
                               Math.min(dp[i - 1][j],      // Delete
                                        dp[i][j - 1]));    // Insert
            }
        }
    }
    return dp[m][n];
}
```

4. Coin Change

Problem: Find minimum number of coins to make up amount.

```
public int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1);
    dp[0] = 0;

    for (int i = 1; i <= amount; i++) {
        for (int coin : coins) {
```

```
                if (coin <= i) {
                    dp[i] = Math.min(dp[i], dp[i - coin] + 1);
                }
            }
        }
        return dp[amount] > amount ? -1 : dp[amount];
    }
```

5. House Robber

Problem: Rob houses along a street (can't rob adjacent houses). Maximize total amount.

```
    public int rob(int[] nums) {
        if (nums.length == 1) return nums[0];

        int prev2 = 0, prev1 = 0;
        for (int num : nums) {
            int curr = Math.max(prev1, prev2 + num);
            prev2 = prev1;
            prev1 = curr;
        }
        return prev1;
    }
```

Interview Tips

Pattern Recognition

- "Maximum/minimum" → likely DP
- "Count ways" → likely DP
- "Can you reach/achieve" → DP or greedy
- Substring/subsequence problems → often 2D DP

Common Mistakes

- Off-by-one errors in array indices
- Forgetting base cases
- Not considering empty input
- Using wrong iteration direction in space-optimized solutions

Space Optimization Template

```
// Convert 2D to 1D when dp[i] only depends on dp[i-1]
// Original: dp[i][j] = f(dp[i-1][j], dp[i-1][j-1])
// Optimized: iterate j backwards to preserve previous row values
for (int i = 1; i <= m; i++) {
    for (int j = n; j >= 1; j--) {  // Backwards!
        dp[j] = f(dp[j], dp[j-1]);
    }
}
```

Time & Space Complexity

Problem Type	Time Complexity	Space Complexity
1D Linear DP	O(n)	O(n) → O(1) optimized
2D Grid DP	O(mn)	O(mn) → O(n) optimized
LCS/Edit Distance	O(mn)	O(mn) → O(min(m,n))
Knapsack	O(nW)	O(W)
LIS	$O(n^2)$ → O(n log n)	O(n)

Most DP problems can be space-optimized by observing that we often only need previous states.

---

10. Graph Algorithms

Common Problem Patterns

1. Number of Islands (BFS/DFS)

Problem: Count number of islands in a 2D grid.

```
public int numIslands(char[][] grid) {
    int count = 0;
    for (int i = 0; i < grid.length; i++) {
        for (int j = 0; j < grid[0].length; j++) {
            if (grid[i][j] == '1') {
                dfs(grid, i, j);
                count++;
```

```
            }
        }
    }
    return count;
}

private void dfs(char[][] grid, int i, int j) {
    if (i < 0 || i >= grid.length || j < 0 || j >= grid[0].length || grid[i][j] != '
    grid[i][j] = '0';
    dfs(grid, i+1, j); dfs(grid, i-1, j); dfs(grid, i, j+1); dfs(grid, i, j-1);
}
```

2. Clone Graph

Problem: Deep clone a graph given a node reference.

```
public Node cloneGraph(Node node) {
    if (node == null) return null;
    Map<Node, Node> map = new HashMap<>();
    return dfs(node, map);
}

private Node dfs(Node node, Map<Node, Node> map) {
    if (map.containsKey(node)) return map.get(node);
    Node clone = new Node(node.val);
    map.put(node, clone);
    for (Node neighbor : node.neighbors) {
        clone.neighbors.add(dfs(neighbor, map));
    }
    return clone;
}
```

3. Course Schedule (Topological Sort)

Problem: Determine if you can finish all courses given prerequisites.

```
public boolean canFinish(int numCourses, int[][] prerequisites) {
    int[] inDegree = new int[numCourses];
    List<List<Integer>> graph = new ArrayList<>();
    for (int i = 0; i < numCourses; i++) graph.add(new ArrayList<>());

    for (int[] pre : prerequisites) {
        graph.get(pre[1]).add(pre[0]);
        inDegree[pre[0]]++;
    }

    Queue<Integer> queue = new LinkedList<>();
    for (int i = 0; i < numCourses; i++) {
```

```
            if (inDegree[i] == 0) queue.offer(i);
        }

        int count = 0;
        while (!queue.isEmpty()) {
            int course = queue.poll();
            count++;
            for (int next : graph.get(course)) {
                if (--inDegree[next] == 0) queue.offer(next);
            }
        }
        return count == numCourses;
    }
```

4. Network Delay Time (Dijkstra)

Problem: Find time for signal to reach all nodes.

```
    public int networkDelayTime(int[][] times, int n, int k) {
        Map<Integer, List<int[]>> graph = new HashMap<>();
        for (int[] t : times) {
            graph.computeIfAbsent(t[0], x -> new ArrayList<>()).add(new int[]{t[1], t[2]
        }

        int[] dist = new int[n + 1];
        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[k] = 0;

        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]);
        pq.offer(new int[]{k, 0});

        while (!pq.isEmpty()) {
            int[] curr = pq.poll();
            int u = curr[0], d = curr[1];
            if (d > dist[u]) continue;

            if (graph.containsKey(u)) {
                for (int[] edge : graph.get(u)) {
                    int v = edge[0], w = edge[1];
                    if (dist[u] + w < dist[v]) {
                        dist[v] = dist[u] + w;
                        pq.offer(new int[]{v, dist[v]});
                    }
                }
            }
        }

        int maxTime = 0;
        for (int i = 1; i <= n; i++) {
            if (dist[i] == Integer.MAX_VALUE) return -1;
            maxTime = Math.max(maxTime, dist[i]);
        }
```

```
        return maxTime;
    }

    ---

    ---

    # 11. Tree Algorithms

    ## Common Problem Patterns
    ### 1. Validate Binary Search Tree

    **Problem:** Check if a binary tree is a valid BST.

    ```java
    public boolean isValidBST(TreeNode root) {
        return validate(root, null, null);
    }

    private boolean validate(TreeNode node, Integer min, Integer max) {
        if (node == null) return true;
        if ((min != null && node.val <= min) || (max != null && node.val >= max)) return
        return validate(node.left, min, node.val) && validate(node.right, node.val, max)
    }
```

2. Lowest Common Ancestor

Problem: Find LCA of two nodes in a binary tree.

```
public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
    if (root == null || root == p || root == q) return root;
    TreeNode left = lowestCommonAncestor(root.left, p, q);
    TreeNode right = lowestCommonAncestor(root.right, p, q);
    if (left != null && right != null) return root;
    return left != null ? left : right;
}
```

3. Binary Tree Maximum Path Sum

Problem: Find maximum path sum (path can start/end at any node).

```
private int maxSum = Integer.MIN_VALUE;

public int maxPathSum(TreeNode root) {
    maxGain(root);
    return maxSum;
}
```
```

```
    private int maxGain(TreeNode node) {
        if (node == null) return 0;
        int left = Math.max(maxGain(node.left), 0);
        int right = Math.max(maxGain(node.right), 0);
        maxSum = Math.max(maxSum, node.val + left + right);
        return node.val + Math.max(left, right);
    }
```

4. Serialize and Deserialize Binary Tree

Problem: Design serialization/deserialization for binary tree.

```
public String serialize(TreeNode root) {
    if (root == null) return "null";
    return root.val + "," + serialize(root.left) + "," + serialize(root.right);
}

public TreeNode deserialize(String data) {
    Queue<String> queue = new LinkedList<>(Arrays.asList(data.split(",")));
    return buildTree(queue);
}

private TreeNode buildTree(Queue<String> queue) {
    String val = queue.poll();
    if (val.equals("null")) return null;
    TreeNode node = new TreeNode(Integer.parseInt(val));
    node.left = buildTree(queue);
    node.right = buildTree(queue);
    return node;
}
```

5. Invert Binary Tree

Problem: Invert a binary tree (mirror).

```
public TreeNode invertTree(TreeNode root) {
    if (root == null) return null;
    TreeNode temp = root.left;
    root.left = invertTree(root.right);
    root.right = invertTree(temp);
    return root;
}
```

---

12. Backtracking

Common Problem Patterns

1. Subsets

Problem: Generate all subsets of a set.

```
public List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    backtrack(nums, 0, new ArrayList<>(), result);
    return result;
}

private void backtrack(int[] nums, int start, List<Integer> path, List<List<Integer>
    result.add(new ArrayList<>(path));
    for (int i = start; i < nums.length; i++) {
        path.add(nums[i]);
        backtrack(nums, i + 1, path, result);
        path.remove(path.size() - 1);
    }
}
```

2. Combination Sum

Problem: Find combinations that sum to target (can reuse elements).

```
public List<List<Integer>> combinationSum(int[] candidates, int target) {
    List<List<Integer>> result = new ArrayList<>();
    backtrack(candidates, target, 0, new ArrayList<>(), result);
    return result;
}

private void backtrack(int[] candidates, int remain, int start,
                       List<Integer> path, List<List<Integer>> result) {
    if (remain == 0) { result.add(new ArrayList<>(path)); return; }
    if (remain < 0) return;

    for (int i = start; i < candidates.length; i++) {
        path.add(candidates[i]);
        backtrack(candidates, remain - candidates[i], i, path, result);
        path.remove(path.size() - 1);
    }
}
```

3. N-Queens

Problem: Place n queens on n×n board so no two attack each other.

```
public List<List<String>> solveNQueens(int n) {
    List<List<String>> result = new ArrayList<>();
    backtrack(n, 0, new int[n], result);
    return result;
}

private void backtrack(int n, int row, int[] queens, List<List<String>> result) {
    if (row == n) {
        result.add(buildBoard(queens, n));
        return;
    }
    for (int col = 0; col < n; col++) {
        if (isValid(queens, row, col)) {
            queens[row] = col;
            backtrack(n, row + 1, queens, result);
        }
    }
}

private boolean isValid(int[] queens, int row, int col) {
    for (int i = 0; i < row; i++) {
        if (queens[i] == col || Math.abs(queens[i] - col) == row - i) return false;
    }
    return true;
}
```

4. Word Search

Problem: Find if word exists in 2D grid.

```
public boolean exist(char[][] board, String word) {
    for (int i = 0; i < board.length; i++) {
        for (int j = 0; j < board[0].length; j++) {
            if (dfs(board, word, i, j, 0)) return true;
        }
    }
    return false;
}

private boolean dfs(char[][] board, String word, int i, int j, int k) {
    if (k == word.length()) return true;
    if (i < 0 || i >= board.length || j < 0 || j >= board[0].length) return false;
    if (board[i][j] != word.charAt(k)) return false;

    char temp = board[i][j];
    board[i][j] = '#';
```

```
        boolean found = dfs(board, word, i+1, j, k+1) || dfs(board, word, i-1, j, k+1) |
                        dfs(board, word, i, j+1, k+1) || dfs(board, word, i, j-1, k+1);
        board[i][j] = temp;
        return found;
    }
```

---

13. Trie & String Algorithms

Common Problem Patterns

1. Implement Trie (Prefix Tree)

Problem: Implement a trie with insert, search, and startsWith.

```
class Trie {
    private TrieNode root = new TrieNode();

    class TrieNode {
        TrieNode[] children = new TrieNode[26];
        boolean isEnd = false;
    }

    public void insert(String word) {
        TrieNode node = root;
        for (char c : word.toCharArray()) {
            if (node.children[c - 'a'] == null) node.children[c - 'a'] = new TrieNod
            node = node.children[c - 'a'];
        }
        node.isEnd = true;
    }

    public boolean search(String word) {
        TrieNode node = searchPrefix(word);
        return node != null && node.isEnd;
    }

    public boolean startsWith(String prefix) {
        return searchPrefix(prefix) != null;
    }

    private TrieNode searchPrefix(String prefix) {
        TrieNode node = root;
        for (char c : prefix.toCharArray()) {
            if (node.children[c - 'a'] == null) return null;
            node = node.children[c - 'a'];
```

```
        }
        return node;
    }
}
```

2. Word Search II

Problem: Find all words from dictionary that exist in board.

```
public List<String> findWords(char[][] board, String[] words) {
    List<String> result = new ArrayList<>();
    TrieNode root = buildTrie(words);

    for (int i = 0; i < board.length; i++) {
        for (int j = 0; j < board[0].length; j++) {
            dfs(board, i, j, root, result);
        }
    }
    return result;
}

private void dfs(char[][] board, int i, int j, TrieNode node, List<String> result) {
    char c = board[i][j];
    if (c == '#' || node.children[c - 'a'] == null) return;
    node = node.children[c - 'a'];
    if (node.word != null) {
        result.add(node.word);
        node.word = null; // Avoid duplicates
    }

    board[i][j] = '#';
    if (i > 0) dfs(board, i-1, j, node, result);
    if (j > 0) dfs(board, i, j-1, node, result);
    if (i < board.length-1) dfs(board, i+1, j, node, result);
    if (j < board[0].length-1) dfs(board, i, j+1, node, result);
    board[i][j] = c;
}
```

3. Longest Common Prefix

Problem: Find longest common prefix among array of strings.

```
public String longestCommonPrefix(String[] strs) {
    if (strs == null || strs.length == 0) return "";

    String prefix = strs[0];
    for (int i = 1; i < strs.length; i++) {
        while (strs[i].indexOf(prefix) != 0) {
```

```
                prefix = prefix.substring(0, prefix.length() - 1);
                if (prefix.isEmpty()) return "";
            }
        }
        return prefix;
    }
```

4. Valid Palindrome

Problem: Check if string is palindrome (alphanumeric only).

```
public boolean isPalindrome(String s) {
    int left = 0, right = s.length() - 1;
    while (left < right) {
        while (left < right && !Character.isLetterOrDigit(s.charAt(left))) left++;
        while (left < right && !Character.isLetterOrDigit(s.charAt(right))) right--;
        if (Character.toLowerCase(s.charAt(left)) != Character.toLowerCase(s.charAt(
            return false;
        }
        left++; right--;
    }
    return true;
}
```

---

14. Math & Bit Manipulation

Common Problem Patterns

1. Single Number

Problem: Find element that appears once (others appear twice).

```
public int singleNumber(int[] nums) {
    int result = 0;
    for (int n : nums) result ^= n;
    return result;
}
```

2. Number of 1 Bits

Problem: Count number of set bits in integer.

```
public int hammingWeight(int n) {
    int count = 0;
    while (n != 0) {
        n &= (n - 1);
        count++;
    }
    return count;
}
```

3. Reverse Bits

Problem: Reverse bits of a 32-bit unsigned integer.

```
public int reverseBits(int n) {
    int result = 0;
    for (int i = 0; i < 32; i++) {
        result = (result << 1) | (n & 1);
        n >>= 1;
    }
    return result;
}
```

4. Power of Two

Problem: Check if n is a power of two.

```
public boolean isPowerOfTwo(int n) {
    return n > 0 && (n & (n - 1)) == 0;
}
```

5. Missing Number

Problem: Find missing number in [0, n].

```
public int missingNumber(int[] nums) {
    int xor = nums.length;
    for (int i = 0; i < nums.length; i++) {
        xor ^= i ^ nums[i];
    }
```

```
        return xor;
    }
```

6. Pow(x, n) - Fast Exponentiation

Problem: Implement pow(x, n).

```
public double myPow(double x, int n) {
    long N = n;
    if (N < 0) { x = 1 / x; N = -N; }
    double result = 1;
    while (N > 0) {
        if ((N & 1) == 1) result *= x;
        x *= x;
        N >>= 1;
    }
    return result;
}
```

---

16. Flow Algorithms

Common Problem Patterns

1. Bipartite Matching

Problem: Given a bipartite graph, find the maximum matching.

```
public int maxBipartiteMatching(int[][] graph) {
    int n = graph.length;
    int m = graph[0].length;

    MaxFlow flow = new MaxFlow(n + m + 2);
    int source = n + m;
    int sink = n + m + 1;

    // Connect source to left nodes
    for (int i = 0; i < n; i++) {
        flow.addEdge(source, i, 1);
    }

    // Connect right nodes to sink
```

```
        for (int j = 0; j < m; j++) {
            flow.addEdge(n + j, sink, 1);
        }

        // Connect left to right based on graph
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                if (graph[i][j] == 1) {
                    flow.addEdge(i, n + j, 1);
                }
            }
        }

        return flow.maxFlow(source, sink);
    }
```

2. Min-Cut Problem

Problem: Find the minimum cut that separates source from sink.

```
public List<List<Integer>> minCut(int source, int sink) {
    int maxFlowValue = maxFlow(source, sink);

    // Run BFS from source on residual graph
    boolean[] reachable = new boolean[n];
    Queue<Integer> queue = new LinkedList<>();
    queue.offer(source);
    reachable[source] = true;

    while (!queue.isEmpty()) {
        int u = queue.poll();

        for (Edge edge : graph.get(u)) {
            if (!reachable[edge.to] && edge.residualCapacity() > 0) {
                reachable[edge.to] = true;
                queue.offer(edge.to);
            }
        }
    }

    // Find cut edges
    List<List<Integer>> cutEdges = new ArrayList<>();
    for (int u = 0; u < n; u++) {
        if (reachable[u]) {
            for (Edge edge : graph.get(u)) {
                if (!reachable[edge.to] && edge.capacity > 0) {
                    cutEdges.add(Arrays.asList(u, edge.to));
                }
            }
        }
    }
```

```
        return cutEdges;
    }
```

3. Project Selection Problem

Problem: Given projects with profits and dependencies, maximize total profit.

```
public int maxProfit(int[] profits, int[][] dependencies) {
    int n = profits.length;
    MaxFlow flow = new MaxFlow(n + 2);
    int source = n;
    int sink = n + 1;

    int totalPositive = 0;

    for (int i = 0; i < n; i++) {
        if (profits[i] > 0) {
            flow.addEdge(source, i, profits[i]);
            totalPositive += profits[i];
        } else {
            flow.addEdge(i, sink, -profits[i]);
        }
    }

    // Add dependency edges with infinite capacity
    for (int[] dep : dependencies) {
        int from = dep[0], to = dep[1];
        flow.addEdge(from, to, Integer.MAX_VALUE);
    }

    return totalPositive - flow.maxFlow(source, sink);
}
```

4. Network Reliability

Problem: Find edge-connectivity of a network.

```
public int edgeConnectivity(int[][] graph) {
    int n = graph.length;
    int minCut = Integer.MAX_VALUE;

    // Try all pairs of vertices as source and sink
    for (int sink = 1; sink < n; sink++) {
        MaxFlow flow = new MaxFlow(n);

        // Add edges from adjacency matrix
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
```

```
                if (graph[i][j] > 0) {
                    flow.addEdge(i, j, graph[i][j]);
                }
            }
        }

        int cut = flow.maxFlow(0, sink);
        minCut = Math.min(minCut, cut);
    }

    return minCut;
}
```

Advanced Variations

Push-Relabel Algorithm

```
public class PushRelabel {
    private int[] height, excess;
    private List<List<Edge>> graph;
    private Queue<Integer> activeVertices;

    public int maxFlow(int source, int sink) {
        initialize(source);

        while (!activeVertices.isEmpty()) {
            int u = activeVertices.poll();

            if (u != source && u != sink && excess[u] > 0) {
                if (!push(u)) {
                    relabel(u);
                    activeVertices.offer(u);
                }
            }
        }

        return excess[sink];
    }

    private boolean push(int u) {
        for (Edge edge : graph.get(u)) {
            if (edge.residualCapacity() > 0 && height[u] == height[edge.to] + 1) {
                int pushFlow = Math.min(excess[u], edge.residualCapacity());

                edge.flow += pushFlow;
                excess[u] -= pushFlow;
                excess[edge.to] += pushFlow;

                if (excess[edge.to] == pushFlow && edge.to != source) {
                    activeVertices.offer(edge.to);
                }
```

```
                return excess[u] == 0;
            }
        }
        return false;
    }

    private void relabel(int u) {
        int minHeight = Integer.MAX_VALUE;

        for (Edge edge : graph.get(u)) {
            if (edge.residualCapacity() > 0) {
                minHeight = Math.min(minHeight, height[edge.to]);
            }
        }

        height[u] = minHeight + 1;
    }
}
```

Interview Tips

Pattern Recognition

- **Maximum flow problems**: Network capacity, resource allocation
- **Minimum cut problems**: Bottleneck identification, network vulnerability
- **Bipartite matching**: Assignment problems, stable marriages
- **Project selection**: Profit maximization with constraints

Common Mistakes

- **Forgetting reverse edges**: Essential for residual graph construction
- **Wrong termination condition**: Must check for augmenting paths correctly
- **Capacity overflow**: Use appropriate data types for large capacities
- **Not handling disconnected components**: Verify graph connectivity

Implementation Variations

```
// Template for flow network construction
class FlowNetwork {
    void addBidirectionalEdge(int u, int v, int capacity) {
        addEdge(u, v, capacity);
        addEdge(v, u, capacity);
    }

    void addDirectionalEdge(int u, int v, int capacity) {
```

```
            addEdge(u, v, capacity);
            addEdge(v, u, 0); // Reverse edge with 0 capacity
        }
    }

    // Template for multiple sources/sinks
    void addSuperSource(int[] sources, int superSource) {
        for (int source : sources) {
            addEdge(superSource, source, Integer.MAX_VALUE);
        }
    }
```

Time & Space Complexity

Algorithm	Time Complexity	Space Complexity	Notes
Ford-Fulkerson	$O(E \times f)$	$O(V + E)$	f = max flow value
Edmonds-Karp	$O(V \times E^2)$	$O(V^2)$	Always polynomial
Dinic's	$O(V^2 \times E)$	$O(V + E)$	Better for unit capacities
Push-Relabel	$O(V^2 \times E)$	$O(V + E)$	Good for dense graphs

---